King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 06

Analysis of Recursive Algorithms

Term 252

Analysis of Recursive Algorithms

- What is a recurrence relation?
- Forming recurrence relations
- Solving Recurrence Relations
- Analysis Of Recursive Factorial method
- Analysis Of Recursive Selection Sort
- Analysis Of Recursive Binary Search
- Analysis Of Recursive Towers of Hanoi Algorithm

What is a recurrence relation?

- A recurrence relation, T(n), is a recursive function of integer variable n.
  - Like all recursive functions, it has both a recursive case and a base case.
- Examples:

$$T(n) = \begin{cases} a & if\ n = 1 \\ 2T\left(\frac{n}{2}\right) + bn + c & if\ n > 1 \end{cases}$$

$$T(n) = \begin{cases} a & if\ n = 1 \\ T(n-1) + bn + c & if\ n > 1 \end{cases}$$

Where a, b and c are integer constants

- The base case of the recurrence relation: the portion of the definition that does not contain T
- The recurrent or recursive case: the portion that contains T.
- Recurrence relations are useful for expressing the running times (i.e., the number of basic operations executed) of recursive algorithms.

Forming Recurrence Relations

- For a given recursive method, the base case and the recursive case of its recurrence relation correspond directly to the base case and the recursive case of the method.
- Example 1: Write the recurrence relation describing the number of multiplication carried out for the factorial method.

```
long factorial (int n) {
   if (n == 0)
      return 1;
   else
      return n * factorial (n - 1);
}
```

$$T(n) = \begin{cases} 0 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

- The base case is reached when n = 0.
  - The number of multiplications is 0, and hence, T(0) = 1.
- When n > 0,
  - The number of multiplications is 1 + T(n-1).
- Therefore, the recurrence relation is:

Forming Recurrence Relations

- Example 2: Write the recurrence relation describing the number of print statements executed for the following method.

```
public void f (int n) {
    if (n > 0) {
        System.out.println(n);
        f(n-1);
    }
}
```

- The base case occues when n = 0.
  - In this case, no print statements are executed, so, T(0) = 0.
- For n > 0,
  - One print statement is executed in addition to the recursive call, , so the total number of executions is 1 + T(n-1).
- What about number of comparisons?
  - When n = 0, one comparison is carried out. So, T(0) = 1.

$$T(n) = \begin{cases} 0 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

Forming Recurrence Relations

- Example 3: Write the recurrence relation describing the number of additions carried out for the following method.

```
public int g(int n) {
   if (n == 1)
      return 2;
   else
      return 3 * g(n / 2) + g( n / 2) + 5;
}
```

$$T(n) = \begin{cases} 0 & n = 1 \\ 2T\left(\frac{n}{2}\right) + 2 & n > 1 \end{cases}$$

- The base case is reached when
  - n = 1 and hence, T(1) = 0
- When n > 1,
  - Hence, the recurrence relation is:

Solving Recurrence Relations

- To solve a recurrence relation T(n) we need to derive a form of T(n) that is not a recurrence relation. Such a form is called a closed form of the recurrence relation.

- There are four methods to solve recurrence relations that represent the running time of recursive methods:
  - Iteration method (unrolling and summing)
  - Substitution method
  - Recursion tree method
  - Master method

- In this course, we will only use the Iteration method.

Solving Recurrence Relations - Iteration method

- Steps:
  - Expand the recurrence
  - Express the expansion as a summation by plugging the recurrence back into itself until you see a pattern.
  - Evaluate the summation
- In evaluating the summation one or more of the following summation formula may be used:

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

$$\sum_{i=0}^{n-1} c = nc$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^{n} \lg(i) \approx n\lg(n)$$

$$\sum_{i=1}^{n} \frac{1}{i} = 1 + \frac{1}{2} + \cdots + \frac{1}{n} \approx ln(n)$$

$$\sum_{i=0}^{n} a^i = 1 + a + a^2 + \cdots + a^n = \frac{a^{n+1} - 1}{a - 1} \qquad a \neq 1$$

Analysis Of Recursive Factorial method

- Example 1: Form and solve the recurrence relation describing the number of multiplications carried out by the factorial method and hence determine its big-O complexity

```
long factorial (int n) {
   if (n == 0)
      return 1;
   else
      return n * factorial (n - 1);
}
```

$$T(n) = \begin{cases} 0 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

$$= n \qquad n \geq 0$$

$$T(n) = T(n-1) + 1$$
$$= (T(n-2) + 1) + 1$$
$$= T(n-2) + 1 + 1$$
$$= (T(n-3) + 1) + 1 + 1$$
$$= T(n-3) + 1 + 1 + 1$$
$$\vdots$$
$$= T(n-k) + k$$
$$\vdots$$

$We\ stop\ when\ (n-k) = 0,$
$i.e., k = n$

$$= T(n-n) + n$$
$$= T(0) + n = 0 + n = O(n)$$

Analysis Of Recursive Towers of Hanoi Algorithm

```
public static void hanoi(int n, char from, char to, char temp){
  if (n == 1)
    System.out.println(from + " --------> " + to);
  else{
    hanoi(n - 1, from, temp, to);
    System.out.println(from + " --------> " + to);
    hanoi(n - 1, temp, to, from);
  }
}
```

- The recurrence relation describing the number of times **the printing statement** is executed for the method Hanoi is:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n-1) + 1 & n > 1 \end{cases}$$

$$= 2^n - 1 \qquad n \geq 1$$

Analysis Of Recursive Towers of Hanoi Algorithm…

$$T(n) = 2T(n-1) + 1$$
$$= 2(2T(n-2) + 1) + 1$$
$$= 22T(n-2) + 2 + 1$$
$$= 2^2(2T(n-3) + 1) + 2 + 1$$
$$= 2^3T(n-3) + 2^2 + 2 + 1$$
$$\vdots$$
$$= 2^kT(n-k) + 2^{k-1} + 2^{k-2} + \cdots + 2 + 1$$

$We\ stop\ when\ (n-k) = 1, i.e., k = n-1$

$$= 2^{n-1}T(n-(n-1)) + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1$$
$$= 2^{n-1}T(1)) + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1$$
$$= 2^{n-1} + 2^{n-2} + 2^{n-3} + \cdots + 2 + 1$$
$$= \sum_{i=0}^{n-1} 2^i = 2^n - 1 = O(2^n)$$

```
public static void hanoi(int n, char from, char to, char temp){
  if (n == 1)
    System.out.println(from + " --------> " + to);
  else{
    hanoi(n - 1, from, temp, to);
    System.out.println(from + " --------> " + to);
    hanoi(n - 1, temp, to, from);
  }
}
```

- The recurrence relation describing the number of times **the printing statement** is executed for the method Hanoi is:

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n-1) + 1 & n > 1 \end{cases}$$

$$= 2^n - 1 \qquad n \geq 1$$
$$= O(2^n)$$

Analysis Of Recursive Binary Search

- The recurrence relation describing the number of **element comparisons** for the method is:

$$T(n) = \begin{cases} 0 & n = 0 \\ T\left(\frac{n}{2}\right) + 1 & n \geq 1 \end{cases}$$

$$= \log n + 1 \qquad n \geq 0$$

```
public int binarySearch (int target, int[] array,
                         int low, int high) {
   if (low > high)
      return -1;
   else {
      int middle = (low + high)/2;
      if (array[middle] == target)
         return middle;
      else if(array[middle] < target)
         return binarySearch(target, array, middle + 1, high);
      else
         return binarySearch(target, array, low, middle - 1);
     }
}
```

Analysis Of Recursive Binary Search

$$T(n) = \begin{cases} 0 & n = 0 \\ T\left(\frac{n}{2}\right) + 1 & n \geq 1 \end{cases}$$

$$T(n) = T(\frac{n}{2}) + 1$$

$$= (T\left(\frac{n}{2^2}\right) + 1) + 1$$

$$= T\left(\frac{n}{2^2}\right) + 2$$

$$= (T\left(\frac{n}{2^3}\right) + 1) + 2$$

$$= T\left(\frac{n}{2^3}\right) + 3$$

$$\vdots$$

$$= T\left(\frac{n}{2^k}\right) + k$$

We should stop when $\left(\frac{n}{2^k}\right) = 0$

However, we can't find k *with respect to n.*

So, we will stop when having one element

So let us stop when $\left(\frac{n}{2^k}\right) = 1$

$$\rightarrow n = 2^k \rightarrow k = \log_2(n)$$

$$T(n) = T\left(\frac{n}{n}\right) + \log_2(n)$$

$$T(n) = T(1) + \log_2(n)$$

$$= 1 + \log_2(n)$$

$$= 1 + \log_2(n)$$

$$= O(\log_2(n))$$

To find T(1):

$$T(1) = T\left(\frac{1}{2}\right) + 1$$

$$T(1) = T(0) + 1 = 1$$

Analysis Of Recursive Selection Sort

- Example 2: Form and solve the recurrence relation describing the **number of element comparisons** (**x[i] > x[k]**) carried out by the selection sort method and hence determine its big-O complexity

$$T(n) = \begin{cases} 0 & n = 1 \\ T(n-1) + n - 1 & n > 1 \end{cases}$$

$$= \frac{(n-1)n}{2} \qquad n \geq 1$$

$$= O(n^2)$$

```
public static void selectionSort(int[] x) {
  selectionSort(x, x.length);}
private static void selectionSort(int[] x, int n) {
  int minPos;
  if (n > 1) {
    maxPos = findMaxPos(x, n - 1);
    swap(x, maxPos, n - 1);
    selectionSort(x, n - 1);
  }
}
private static int findMaxPos (int[] x, int j) {
  int k = j;
  for(int i = 0; i < j; i++)
    if(x[i] > x[k])  k = i;
  return k;
}
private static void swap(int[] x, int maxPos, int n) {
  int temp=x[n]; x[n]=x[maxPos]; x[maxPos]=temp;
}
```

Analysis Of Recursive Selection Sort

$$T(1) = 0$$

$$T(n) = T(n-1) + (n-1)$$

$$= (T(n-2) + (n-2)) + (n-1)$$

$$= T(n-2) + (n-2) + (n-1)$$

$$= \big(T(n-3) + (n-3)\big) + (n-2) + (n-1)$$

$$= T(n-3) + (n-3) + (n-2) + (n-1)$$

$$\vdots$$

$$= T(n-k) + \sum_{j=1}^{k} (n-j)$$

$$\vdots$$

$$T(n) = \begin{cases} 0 & n = 1 \\ T(n-1) + n - 1 & n > 1 \end{cases}$$

$$\sum_{j=1}^{n-1} (n-j) = \sum_{j=1}^{n-1} j$$

We stop when $(n-k) = 1 \rightarrow k = n-1$

$$= T(n-(n-1)) + \sum_{j=1}^{n-1} (n-j)$$

$$= T(1) + \sum_{j=1}^{n-1} (n-j)$$

$$= 0 + n(n-1) - \frac{(n-1)n}{2} = \frac{(n-1)n}{2}$$

$$= O(n^2)$$

Analysis Of Recursive Fibonacci

$$T(n) = \begin{cases} 0 & n \ in \ \{0,1\} \\ T(n-1) + T(n-2) + 1 & n > 2 \end{cases}$$

$$T(n) \leq \begin{cases} 0 & n \ in \ \{0,1\} \\ 2T(n-1) + 1 & n > 2 \end{cases}$$

```
static int fib( int n){
        if (n == 0 )
                return 0;
        if  ( n == 1 )
                return 1;
        else
                return fib(n-1)+fib(n-2);
}
```